DEVSOLUTIONS S.A.S.

Documento de Arquitectura

Bar Inventory APP

Plataforma de gestión integral · Café Colombia Bar

INFORMACIÓN DEL DOCUMENTO

Versión	2.1
Fecha	29 de octubre de 2026
Cliente	Café Colombia Bar — Sedes Galería, Zona T y Modelia
Equipo	Johan Camilo Pedraos · Sofía Pedraza · Jose Florián
Repositorio	github.com/johanyopasa03-droid/App-Inventory-bar

Versión del documento

Versión	Fecha	Autor	Descripción
1.0	30/09/2026	DevSolutions S.A.S.	Creación del documento de arquitectura.
2.0	01/10/2026	DevSolutions S.A.S.	Se ajusta a la implementación real del repositorio: tecnologías (Django 5.2 + Django REST Framework, React 18 + Vite, MySQL 8), modelo de arquitectura, diagramas y modelo entidad-relación generado desde las migraciones de Django.
2.1	29/10/2026	DevSolutions S.A.S.	Diagrama de despliegue con flechas bidireccionales según el flujo de comunicación; la seguridad OWASP se limita a las prácticas y restricciones de las historias de usuario; se elimina el respaldo de la base de datos.

Tabla de contenido

Versión del documento....2
1. Introducción....3
2. Tecnologías....4
3. Diagrama de arquitectura....5
    3.1 Modelo de arquitectura....5
4. Diagrama de componentes....6
5. Diagrama de despliegue....7
6. Modelo entidad-relación....8
7. Listado de buenas prácticas....9
8. Diccionario de términos....10

1. Introducción

Este documento describe la arquitectura de **Bar Inventory APP**, la aplicación web que DevSolutions S.A.S. desarrolla para Café Colombia Bar. El sistema centraliza la operación de las sedes Galería, Zona T y Modelia: inventario por sede, mesas, pedidos, pagos, facturación, reportes y auditoría.

La aplicación tiene tres roles. El **Administrador** gestiona todas las sedes y la configuración del sistema. El **Cajero** cobra los pedidos y maneja el inventario de su sede. El **Mesero** toma los pedidos en las mesas de su sede. La arquitectura responde a las 30 historias de usuario del backlog (HU01–HU30) y a los requisitos de la propuesta comercial: respuesta menor a 2 segundos, al menos 20 usuarios simultáneos, seguridad basada en OWASP limitada a las prácticas y restricciones que definen las historias de usuario, cierre de sesión a los 3 minutos de inactividad e interfaz web responsive en inglés para Chrome.

A la fecha de esta versión están implementados los Sprints 1 a 3 (HU01–HU21): autenticación, sedes, usuarios, catálogo, proveedores, inventario, mesas y pedidos. Pagos, facturación, reportes y la consulta del log de auditoría (HU22–HU30) corresponden a los Sprints 4 y 5.

2. Tecnologías

Capa	Tecnología	Uso en el proyecto
Frontend	JavaScript · React 18 · Vite 5	Aplicación de una sola página (SPA) con la interfaz en inglés.
Estilos	Tailwind CSS 3	Diseño responsive para PC, tablet y celular.
Navegación y HTTP	React Router 6 · Axios	Rutas protegidas por rol y cliente HTTP que adjunta el token JWT.
Backend	Python 3.13 · Django 5.2	Lógica de negocio, modelos, migraciones y panel de administración.
API	Django REST Framework 3.16	Servicios REST en /api/ con serializers, vistas y paginación.
Autenticación	djangorestframework-simplejwt 5.5	Tokens JWT de acceso (3 min) y refresco, con lista negra al cerrar sesión.
Base de datos	MySQL 8 (producción) · SQLite (desarrollo)	Persistencia; el motor se elige con DB_ENGINE en el archivo .env.
Conector	PyMySQL 1.1	Conexión de Django con MySQL.
Archivos	Pillow 11	Carga y validación de imágenes de producto.
Configuración	python-dotenv · django-cors-headers	Variables de entorno y acceso del frontend a la API.
Servidor (despliegue)	Nginx · Gunicorn · Docker Compose	Publicación en un VPS Linux.
Herramientas	Git · GitHub · Postman	Control de versiones (rama por HU) y pruebas de la API.

3. Diagrama de arquitectura

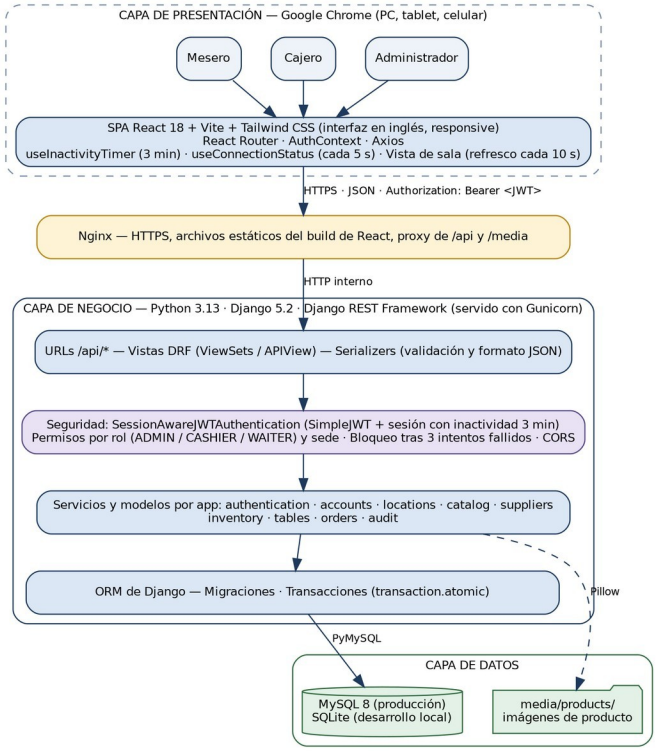


Figura 1. Arquitectura en capas de Bar Inventory APP.

3.1 Modelo de arquitectura

El sistema usa un modelo **cliente-servidor en tres capas** comunicadas por una **API REST**:

- **Capa de presentación:** SPA en React que corre en el navegador. Solo muestra información y llama a la API; no contiene reglas de negocio.
- **Capa de negocio:** backend en Django REST Framework organizado como un monolito modular: una aplicación de Django por módulo funcional, cada una con sus modelos, serializers, vistas, URLs, servicios y pruebas.
- **Capa de datos:** MySQL, a la que se accede únicamente mediante el ORM de Django y sus migraciones.

Dentro de Django se aplica el patrón Modelo–Vista propio de DRF: el modelo define los datos, el serializer valida y transforma a JSON y la vista atiende la petición HTTP. La lógica que toca varias tablas (por ejemplo, descontar stock al agregar un producto) va en archivos services.py y se ejecuta dentro de transacciones.

La sesión se controla en ambos lados: el frontend cierra la sesión tras 3 minutos sin actividad y el backend rechaza el token si la sesión superó ese tiempo. La vista de sala se actualiza consultando la API cada 10 segundos y la conexión se verifica cada 5 segundos.

4. Diagrama de componentes

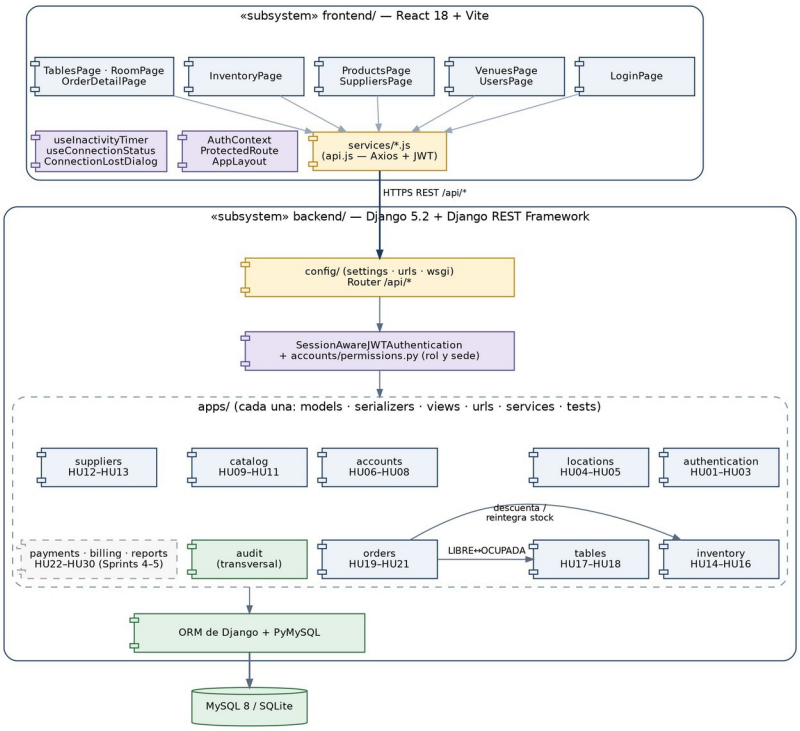


Figura 2. Componentes del frontend y del backend.

Componente	Responsabilidad
authentication	Inicio y cierre de sesión, bloqueo tras 3 intentos fallidos, control de inactividad y pérdida de conexión (HU01–HU03).
locations	Creación, edición e inactivación de sedes (HU04–HU05).
accounts	Usuarios con rol (ADMIN, CASHIER, WAITER) y sede asignada; permisos por rol (HU06–HU08).
catalog	Catálogo único de productos con imagen y precios (HU09–HU11).
suppliers	Directorio de proveedores y su asociación con productos (HU12–HU13).
inventory	Stock por sede, entradas de mercancía y movimientos de inventario (HU14–HU16).
tables	Mesas por sede y vista de sala con estado LIBRE / OCUPADA (HU17–HU18).
orders	Apertura de pedidos, ítems, cancelación y envío a caja (HU19–HU21).
audit	Registro inmutable de eventos usado por todos los módulos.
payments · billing · reports	Pendientes: pagos, facturas, reportes y consulta del log (HU22–HU30).

5. Diagrama de despliegue

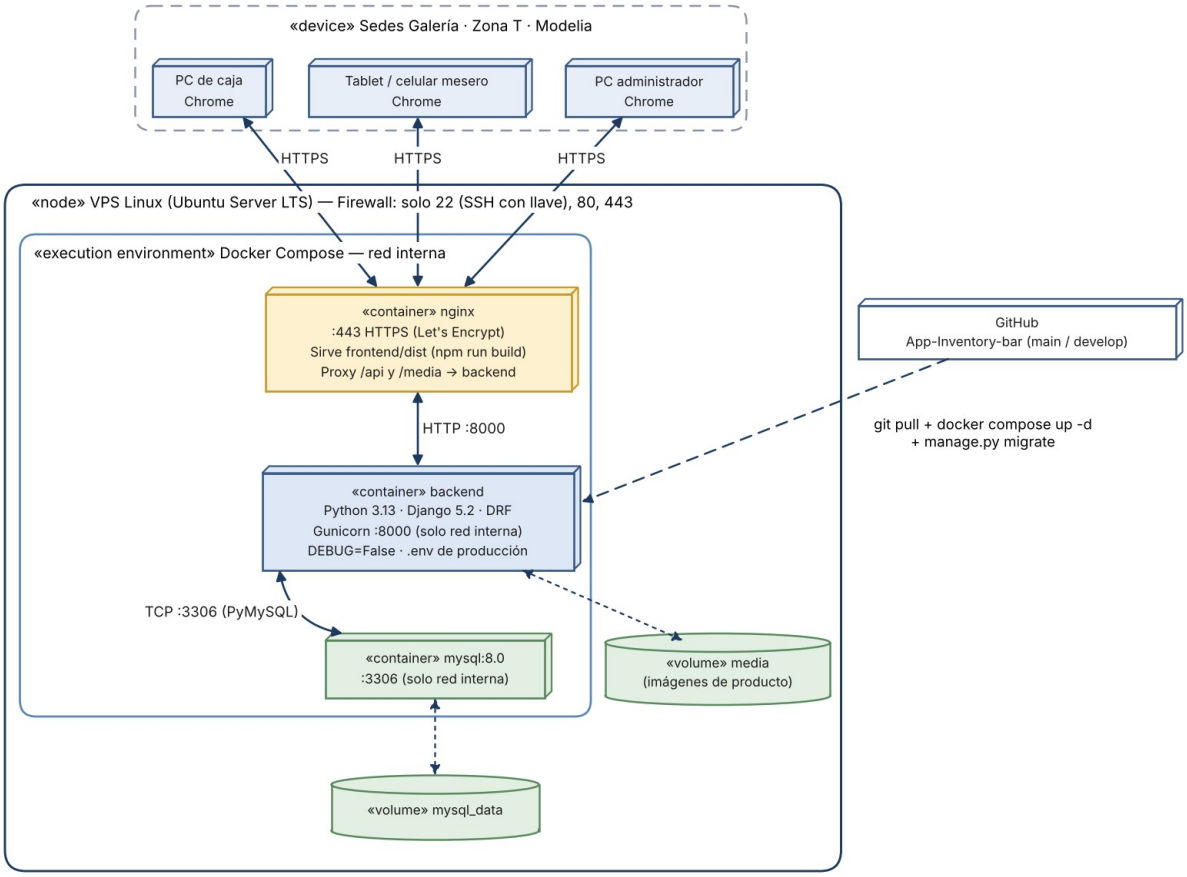


Figura 3. Despliegue en un VPS Linux con Docker Compose.

La aplicación se publica en un **VPS Linux** con tres contenedores: **nginx** sirve el build de React y redirige /api y /media al backend; **backend** ejecuta Django con Gunicorn; **mysql** guarda los datos en un volumen persistente. Desde internet solo se exponen los puertos 80 y 443.

Las flechas dobles indican comunicación de ida y vuelta: el navegador envía la petición a nginx y recibe la respuesta, nginx reenvía las llamadas de la API al backend y devuelve su resultado, y el backend consulta MySQL y recibe los datos. Las líneas punteadas dobles representan lectura y escritura sobre los volúmenes. El despliegue desde GitHub va en un solo sentido, hacia el servidor.

6. Modelo entidad-relación

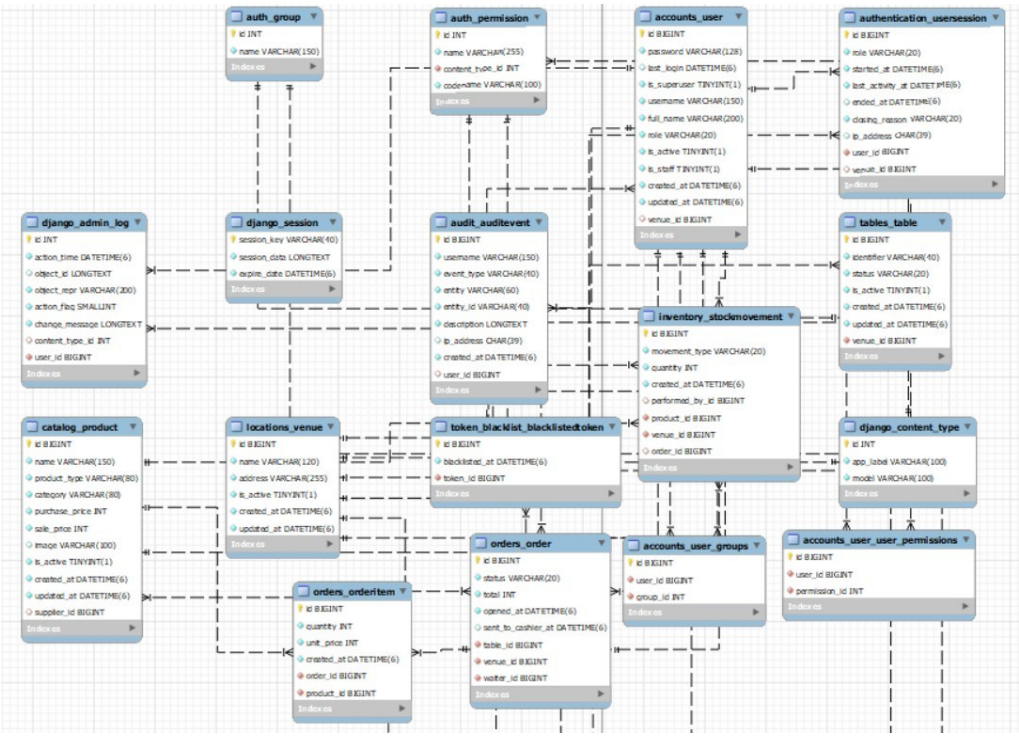


Figura 4. Modelo entidad-relación de la base bar_inventory (MySQL Workbench, ingeniería inversa).

El modelo se generó a partir de las migraciones de Django ejecutadas en MySQL 8. Además de las 12 tablas de negocio descritas abajo, la base contiene las tablas internas de Django (permisos, grupos, tipos de contenido, sesiones, registro de administración, migraciones y lista negra de tokens JWT). El script completo para crear la base está en el archivo bar_inventory_mysql.sql.

Tabla	Descripción
locations_venue	Sedes del negocio.
accounts_user	Usuarios con rol y sede asignada; contraseña cifrada por Django.
authentication_loginattempt	Intentos de inicio de sesión (bloqueo tras 3 fallidos).
authentication_usersession	Sesiones con última actividad y motivo de cierre.
suppliers_supplier	Proveedores.
catalog_product	Productos del catálogo, con su proveedor principal.
inventory_stock	Stock de cada producto en cada sede (único por sede y producto).
inventory_stockmovement	Entradas, descuentos por pedido y reintegros de inventario.
tables_table	Mesas por sede (identificador único dentro de la sede).
orders_order	Pedidos (OPEN, IN_CASHIER, CANCELLED); uno OPEN por mesa.
orders_orderitem	Productos del pedido con el precio unitario congelado.
audit_auditevent	Log de auditoría de solo inserción.

7. Listado de buenas prácticas

Las prácticas 1 a 7 son los controles de seguridad (OWASP) que se aplican en el proyecto. Se limitan a lo que exigen las historias de usuario; la columna HU indica de dónde sale cada una.

#	Práctica	Cómo se aplica	HU
1	Contraseñas cifradas	La contraseña se guarda cifrada (hash PBKDF2 de Django) al crear el usuario y al cambiarla; nunca en texto plano.	HU06, HU07
2	Acceso solo a usuarios autorizados	No hay autoregistro: las cuentas las crea el Administrador y solo los usuarios activos pueden iniciar sesión.	HU01, HU06, HU08
3	Mensaje de error genérico	Si el login falla, el mensaje no indica cuál campo es incorrecto.	HU01
4	Control de acceso por rol y sede	Cada vista valida en el servidor el rol y la sede del usuario. El cajero solo ve su sede y solo el Administrador entra a Auditoría.	HU06, HU15, HU26, HU28, HU30
5	Sesión invalidada en el servidor	El cierre por 3 minutos de inactividad, el cierre manual y la inactivación del usuario invalidan la sesión en el servidor, no solo en el frontend.	HU02, HU03, HU08
6	Validación de datos	El servidor valida nombres únicos, valores numéricos enteros, stock suficiente y que la fecha inicial no sea mayor que la final.	HU04, HU05, HU09, HU12, HU16, HU29
7	Registros inmutables y trazables	El log de auditoría no permite modificar ni eliminar registros y guarda usuario, sede, fecha y hora. Un pedido enviado a caja no admite cambios.	HU02, HU21, HU30
8	Borrado lógico	Sedes, usuarios, productos y mesas se inactivan, no se eliminan; su historial se conserva.	HU05, HU08, HU10, HU17
9	Transacciones	Las operaciones sobre varias tablas usan transaction.atomic.	—
10	Migraciones versionadas	El esquema solo cambia mediante migraciones de Django guardadas en el repositorio.	—
11	Pruebas por historia	Cada HU tiene su archivo de pruebas (tests/test_huXX_*.py).	—
12	Git Flow por HU	Rama feature/HU-XX por historia, commits con el identificador y merge a develop.	—

8. Diccionario de términos

Término	Definición
Administrador (ADMIN)	Rol con acceso a todas las sedes, la configuración, los reportes y la auditoría.
API REST	Servicios web que el frontend usa para consultar y guardar información.
Auditoría	Registro de las acciones de los usuarios: quién, qué, cuándo y en qué sede.
Cajero (CASHIER)	Rol que cobra los pedidos, genera facturas y maneja el inventario de su sede.
Catálogo	Lista única de productos, con el mismo precio en todas las sedes.
Django	Framework web de Python usado en el backend.
Django REST Framework	Extensión de Django para construir la API REST.
Docker	Herramienta que empaqueta cada servicio en un contenedor.
JWT	Token firmado que identifica al usuario en cada petición a la API.
Migración	Archivo de Django que crea o modifica tablas de la base de datos de forma versionada.
Mesero (WAITER)	Rol que crea los pedidos en las mesas y los envía a caja.
MySQL	Sistema de base de datos relacional usado en producción.
Nginx	Servidor web que recibe las conexiones HTTPS y las dirige al backend.
ORM	Capa que traduce los modelos de Python a tablas y consultas SQL.
OWASP	Organización que publica los principales riesgos de seguridad web. En este proyecto se aplican solo los controles que piden las historias de usuario (sección 7).
Pedido	Orden de productos de una mesa: OPEN, IN_CASHIER o CANCELLED.
React	Biblioteca de JavaScript con la que se construye la interfaz.
Responsive	Diseño que se adapta a PC, tablet y celular.
Sede (Venue)	Punto de operación del bar: Galería, Zona T o Modelia.
Serializer	Componente de DRF que valida los datos y los convierte a JSON.
SPA	Aplicación web que carga una vez y cambia de vista sin recargar la página.
SQLite	Base de datos en archivo usada en desarrollo local (db.sqlite3).
Stock	Cantidad disponible de un producto en una sede, en unidades enteras.
VPS	Servidor privado virtual en la nube donde se aloja la aplicación.